

TEMA N° 2



TEMA 2 PROMPT ESTRUCTURADO PARA RESOLUCIÓN LÓGICA



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dominar** el diseño y la redacción de prompts altamente estructurados, aplicando restricciones lógicas y reglas de negocio precisas para automatizar procesos tecnológicos orientados a los emprendimientos locales de la población de Pailón.
2. **Construir** instrucciones avanzadas que permitan a los modelos de Inteligencia Artificial generar, validar y traducir algoritmos en pseudocódigo y estructuras de diagramas de flujo claros, optimizando el tiempo de desarrollo de software.
3. **Adaptar** con precisión milimétrica las respuestas de cualquier modelo de lenguaje a formatos técnicos estandarizados como Markdown y CSV, garantizando que la salida sea directamente utilizable en sistemas informáticos y hojas de cálculo sin necesidad de limpieza manual de datos.



PRÁCTICA



PREGUNTA DETONANTE

¿Alguna vez has intentado pedirle ayuda a una Inteligencia Artificial para resolver un problema de lógica de programación y te ha devuelto un código confuso, lleno de explicaciones innecesarias, o estructurado para una gran empresa internacional que no se adapta en absoluto a las necesidades de un negocio real en tu comunidad?

EL RETO TECNOLÓGICO EN PAILÓN

Don Rigoberto es un emprendedor local muy conocido en el **Barrio Central Fátima**, ubicado al frente de la plaza principal e iglesia de Pailón. Él es propietario de la distribuidora de abarrotes más grande de la zona, encargada de abastecer a pequeñas tiendas en barrios como **María Belén, Jardines de Pailón, San Antonio y Ovidio Barberi**. Con la reciente inauguración del nuevo módulo educativo del **CEA Herman Ortiz Camargo** en el **Barrio Saó**, Don Rigoberto decidió contratar a los estudiantes de la carrera de Sistemas Informáticos para resolver un gran problema logístico.

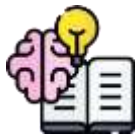
Los camiones repartidores de la distribuidora recogen la mercadería de los vagones de carga en la estación de trenes del **Barrio 24 de Septiembre**. El flujo logístico actual de Don Rigoberto exige cumplir con las siguientes reglas de negocio:

1. Si el pedido es solicitado por una tienda del **Barrio Saó** o del **Barrio 27 de Mayo** (cerca del coliseo municipal) y supera los 500 Bolivianos (Bs.), se aplica automáticamente un descuento del 7% por cercanía y se prioriza la entrega en el mapa de ruta antes de que colapsen las calles por eventos deportivos.
2. Si el pedido va hacia zonas más alejadas como el **Barrio San Exposito** o el **Barrio Las Misiones**, el costo de envío se incrementa en un 12% si el peso total supera los 200 kilogramos, debido al estado de las vías en época de lluvia.
3. Para pedidos del **Barrio Residencial Norte**, solo se permite la entrega en horarios matutinos debido a normativas vecinales de cuidado de parques infantiles.



Los estudiantes del CEA intentaron usar una Inteligencia Artificial generativa para construir el algoritmo de facturación y despacho. Escribieron un prompt simple en el chat: *"Hazme un programa para controlar el despacho de un negocio en Pailón"*. La IA les devolvió un sistema de software genérico redactado en lenguaje de programación C++, estructurado en dólares, que requería librerías de servidores web internacionales y que ignoraba por completo la geografía y reglas de los barrios de Pailón. El código arrojaba errores lógicos de sintaxis, no calculaba los descuentos locales y confundía las prioridades de entrega.

Como futuros profesionales, los estudiantes se dieron cuenta de que el problema no era la Inteligencia Artificial, sino la forma en que estructuraron la orden. La falta de especificaciones técnicas, restricciones operativas y ejemplos prácticos provocó una respuesta inutilizable. Este escenario real nos demuestra la necesidad imperiosa de aprender a diseñar prompts estructurados para la resolución lógica de problemas informáticos comunitarios.



TEORÍA



2.1. DISEÑO DE PROMPTS PARA GENERAR Y VALIDAR ALGORITMOS PSEUDOCÓDIGO

El pseudocódigo es una herramienta fundamental para el Técnico en Sistemas Informáticos, ya que permite expresar la solución de un problema mediante un lenguaje intermedio entre el habla humana y el código de programación puro. Cuando interactuamos con una Inteligencia Artificial para que genere pseudocódigo, la precisión de la estructura del prompt determina la calidad del algoritmo resultante. Un prompt mal diseñado induce a la IA a "alucinar", inventando palabras clave inexistentes o mezclando sintaxis de diferentes lenguajes.

Para diseñar un prompt de generación y validación eficaz, se debe aplicar una estructura de ingeniería de instrucciones que delimite el rol del modelo, el contexto de operación, la tarea exacta y el estándar de salida (por ejemplo, compatibilidad con el software educativo PSeInt).



A continuación, se presenta una plantilla técnica de prompt estructurado diseñada para resolver el control de existencias en una tienda del **Barrio San Antonio**:

Plaintext

```
[ROL] Actúa como un experto docente de programación y validador de algoritmos de software.[CONTEXTO] Estamos programando un sistema para una tienda de repuestos de motocicletas en el Barrio San Antonio de Pailón.[TAREA] Genera un algoritmo en pseudocódigo estricto compatible con PSeInt para evaluar el stock de un repuesto específico.[REGLAS DE NEGOCIO]- Entrada: stock_actual (entero), cantidad_solicitada (entero).- Si la cantidad_solicitada es mayor al stock_actual, mostrar mensaje "Error: Stock insuficiente en Barrio San Antonio".- Si es menor o igual, restar la cantidad al stock y mostrar "Venta aprobada. Stock remanente: [valor]".[FORMATO] Devuelve únicamente el bloque de código del pseudocódigo, comentando cada línea de forma didáctica. No agregues introducciones ni saludos.
```

Cuando la IA procesa este nivel de estructura, el resultado es limpio, preciso y libre de errores semánticos, tal como se aprecia en el siguiente algoritmo generado bajo este estándar:

Delphi

```
// Algoritmo para el control de inventario del Barrio San Antonio
Algoritmo ControlInventarioSanAntonio Definir stockActual, cantidadPedido Como Entero;
// Declara las variables de tipo entero para el stock y el pedido Definir nombreProducto Como Cadena;
```



```
// Declara la variable de tipo texto para el nombre de la mercadería
stockActual <- 45;
// Inicializa el stock disponible en la tienda con un valor fijo de 45 unidades
Escribir "
Ingrese el nombre del producto solicitado por el cliente:";
// Muestra un mensaje en pantalla solicitando el nombre del artículo      Leer
nombreProducto;
// Captura el texto ingresado por el usuario y lo guarda en la variable
nombreProducto      Escribir "
Ingrese la cantidad de unidades que requiere el pedido:";
// Solicita en pantalla la cantidad de piezas deseadas      Leer cantidadPedido;
// Captura el valor numérico ingresado por teclado y lo almacena en
cantidadPedido      Si cantidadPedido <= stockActual Entonces // Evalúa si hay
suficiente inventario disponible para cubrir el pedido solicitado
stockActual <- stockActual - cantidadPedido;
// Resta la cantidad del pedido al stock actual para actualizar el inventario
Escribir "
Pedido aprobado para despacho hacia el Barrio San Antonio.";
// Muestra confirmación de que la transacción es exitosa      Escribir "
Stock restante de ", nombreProducto, ": ", stockActual;
// Imprime en pantalla el saldo actualizado del inventario      Sino // Bloque que
se ejecuta si la condición lógica anterior resulta falsa por falta de mercadería
Escribir "
Error: Stock insuficiente en la distribuidora de Pailón.";
// Informa al usuario que no se puede procesar por falta de stock      FinSi //
Finaliza la estructura de control condicional de manera correctaFinAlgoritmo //
Termina la ejecución global de todo el algoritmo de control
```

2.2. PROMPTS PARA CREACIÓN DE ESTRUCTURAS DE DIAGRAMAS DE FLUJO

Un diagrama de flujo representa visualmente la secuencia lógica de los pasos de un algoritmo. Tradicionalmente, la IA ha tenido dificultades para entregar gráficos directos debido a que su naturaleza es puramente textual. Sin embargo, un Técnico puede sortear esta barrera solicitando a la IA que construya la estructura del diagrama utilizando lenguajes de descripción de gráficos basados en texto, como **Mermaid.js**, o mediante esquemas tabulares tabulados hiper-detallados.

Para que la IA dibuje un flujo de control impecable, el prompt debe proveer la sintaxis básica esperada y especificar cómo se deben mapear las decisiones lógicas (rombos) y los procesos (rectángulos). Si analizamos un sistema de bombeo de agua automatizado para las huertas agrícolas ubicadas en los límites del **Barrio Oto Waver**, el prompt estructural debe exigir la correcta jerarquía de nodos.



Observemos el diseño de las instrucciones técnicas necesarias para obtener la arquitectura de un diagrama de flujo analítico:

Plaintext

[ROL] Actúa como un Arquitecto de Software Senior especializado en modelado de procesos lógicos.[TAREA] Crea la estructura de un diagrama de flujo detallado utilizando la sintaxis de Mermaid.js.[PROCESO A MODELAR] Sistema de activación de riego automático en el Barrio Oto Waver.[REGLAS LOGICAS]- El sistema lee un sensor de humedad_suelo (porcentaje de 0 a 100).- Si humedad_suelo es menor a 40%, se activa la electrobomba y se muestra " Riego Iniciado en Oto Waver".- Si la humedad_suelo es mayor o igual a 40%, la electrobomba se apaga o se mantiene apagada, mostrando " Suelo Óptimo".[INSTRUCCIÓN DE ESTILO] Usa nodos con formas correctas: rectángulos para acciones, rombos para condiciones. Incluye comentarios aclaratorios en la estructura.

El motor de Inteligencia Artificial interpretará esta orden estructurada y generará un bloque de código limpio que puede ser renderizado en cualquier visor de diagramas moderno:

Fragmento de código

```
graph TD
    %% Define el punto de inicio del flujo automatizado en Oto Waver
    A([Inicio del Sistema Riego Oto Waver]) --> B[Leer Porcentaje de Humedad del Suelo];
    %% Almacena y procesa el dato recibido por los sensores de la huerta
    B --> C{¿Humedad es Menor al 40%?};
    %% Flujo lógico condicional en caso de que el suelo esté muy seco
    C -- Sí --> D[Activar Electrobomba de Agua];
    D --> E[Mostrar Mensaje: Riego Iniciado en Oto Waver];
    %% Flujo lógico condicional en caso de que el suelo tenga agua suficiente
    C -- No --> F[Desactivar/Mantener Apagada La Electrobomba];
    F --> G[Mostrar Mensaje: Suelo Óptimo en La Parcela];
    %% Ambos caminos convergen para la finalización del ciclo de lectura
    E --> H([Fin del Ciclo de Control]);
    G --> H;
```

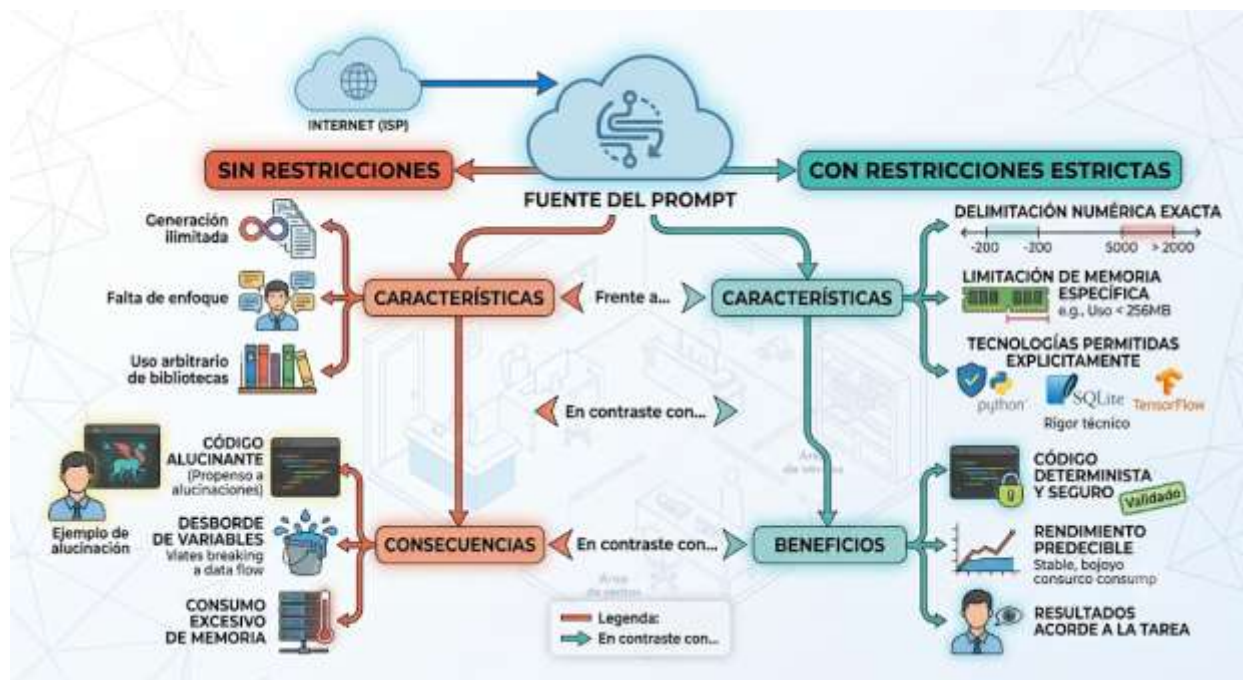
2.3. DEFINICIÓN DE RESTRICCIONES ERICTAS EN EL PROMPT PARA PROGRAMACIÓN

Uno de los errores más frecuentes cometidos por programadores novatos es dar demasiada libertad creativa a los modelos de lenguaje. Cuando una IA programa sin límites explícitos, introduce funciones obsoletas, usa variables globales ineficientes o cambia caprichosamente las



tecnologías del entorno. En el desarrollo profesional, la delimitación de fronteras lógicas infranqueables se conoce como **Prompting de Restricción Negativa y Positiva**.

Para asegurar la estabilidad de un software que registre deportistas y controle los ingresos económicos en los campeonatos nocturnos organizados en el coliseo municipal del **Barrio 27 de Mayo**, un Técnico debe parametrizar el prompt usando imperativos categóricos categóricos (como "PROHIBIDO", "EXCLUSIVAMENTE", "NUNCA").



Analicemos la estructura de un prompt técnico blindado contra fallas lógicas de alcance:

Plaintext

```
[CONTEXT0] Sistema de cobro de entradas del Coliseo del Barrio 27 de Mayo.[TECNOLOGÍA] Lenguaje JavaScript moderno (ES6+).[RESTRICCIONES ERICTAS - OBLIGATORIO CUMPLIR]1. NUNCA utilices librerías externas como jQuery o Axios. Usa código nativo puro (Vanilla JS).2. QUEDA PROHIBIDO el uso de la palabra clave 'var' para declarar variables; utiliza exclusivamente 'let' y 'const' para asegurar el alcance de bloque.3. El formato de la moneda debe ser validado estrictamente en Bolivianos (Bs.). Cualquier valor de entrada menor o
```



igual a cero debe disparar una excepción lógica de tipo Error.4. Las funciones deben incluir manejo de errores explícito mediante bloques `try/catch`. [OUTPUT] Devuelve solo la función de cálculo del total recaudado, libre de explicaciones textuales.

El código generado bajo estas directrices se adhiere de forma milimétrica al estándar de desarrollo seguro requerido por los entornos modernos, eliminando cualquier código basura:

JavaScript

```
// Función estricta para calcular la recaudación de entradas del Coliseo 27 de Mayo
const calcularRecaudacionColiseo = (cantidadEntradas, precioUnitarioBs) => {
  // Validación inicial: La cantidad de tickets vendidos no puede ser negativa ni
  // cero
  if (cantidadEntradas <= 0) {
    throw new Error("
    La cantidad de entradas vendidas debe ser mayor a cero.");
  }
  // Dispara error lógico
  // Validación de moneda: el precio de la entrada debe ser un número real
  // positivo en Bs.
  if (precioUnitarioBs <= 0) {
    throw new Error("
    El precio de la entrada en Bs. debe ser un monto financiero válido.");
  }
  // Detiene la ejecución
  try {
    // Cálculo del monto bruto multiplicando los dos parámetros de entrada validados
    const totalRecaudado = cantidadEntradas * precioUnitarioBs;
    // Variable de bloque inmutable // Retorna el resultado final
    // asegurando que el flujo termine correctamente
    return totalRecaudado;
    // Devuelve el valor numérico exacto en Bolivianos
  }
  catch (error) {
    // Captura cualquier fallo inesperado durante la operación aritmética interna
    console.error("
    Fallo crítico en el procesamiento del cálculo aritmético: " + error.message);
    // Registra el error
  }
};
// Cierre de la constante funcional modular
```

2.4. INYECCIÓN DE EJEMPLOS DE ENTRADA/SALIDA (I/O) EN EL PROMPT

La técnica avanzada de inyectar ejemplos dentro de una instrucción se denomina **Few-Shot Prompting**. Consiste en mostrarle activamente a la Inteligencia Artificial el formato preciso y los datos lógicos de ejemplos previos (Entrada -> Salida), obligándola a comprender el patrón matemático subyacente antes de resolver el nuevo caso propuesto. Es la herramienta definitiva para erradicar las respuestas imprecisas.



Aliquemos este enfoque a las líneas de mototaxis que operan transportando comerciantes desde la feria del **Barrio Central Fátima** hasta los nuevos asentamientos del **Barrio Las Misiones**. El costo del pasaje varía según la distancia, el estado climático y la carga que lleve el pasajero en el vehículo.

Para estructurar correctamente los ejemplos de Entrada/Salida dentro del prompt informático, se debe seguir la siguiente metodología sintáctica:

Plaintext

```
[ROL] Eres un motor de procesamiento de datos lógicos para sistemas de
transporte en Chiquitos.[TAREA] Calcula la tarifa final del servicio de mototaxi
basándote estrictamente en los patrones de los ejemplos provistos.[EJEMPLO 1]-
Entrada: {
Origen: "
Central Fátima", Destino: "
Barrio Saó", Carga: "
No", Lluvia: "
No" }
- Salida: {
TarifaBs: 10, Prioridad: "
Estándar", Mensaje: "
Ruta pavimentada segura" }
[EJEMPLO 2]- Entrada: {
Origen: "
Central Fátima", Destino: "
Las Misiones", Carga: "
Sí", Lluvia: "
Sí" }
- Salida: {
TarifaBs: 25, Prioridad: "
Alta", Mensaje: "
Ruta crítica por desvío de barro" }
[EJERCICIO A RESOLVER]- Entrada: {
Origen: "
Central Fátima", Destino: "
Barrio San Expedito", Carga: "
Sí", Lluvia: "
No" }
[FORMATO DE SALIDA] Devuelve la estructura JSON correspondiente que complete el
patrón del ejercicio.
```



Gracias a los datos inyectados, la Inteligencia Artificial no necesita especular ni inventar reglas complejas; simplemente deduce matemáticamente el comportamiento del sistema de tarifas locales y devuelve una respuesta estructurada perfecta:

JSON

```
{  
  "  
  TarifaBs": 18, "  
  Prioridad": "  
  Media", "  
  Mensaje": "  
  Ruta de tierra transitable con carga pesada controlada"}  
}
```

2.5. ADAPTACIÓN DEL OUTPUT DE IA A FORMATOS ESPECÍFICOS (MARKDOWN, CSV)

En el ejercicio cotidiano del Técnico en Sistemas Informáticos, los datos generados por una Inteligencia Artificial deben interactuar con otras aplicaciones de software. Si la IA entrega un reporte en texto plano desordenado, este no podrá ser cargado en una base de datos SQL o en una planilla de Microsoft Excel. Por ende, es obligatorio aprender a forzar el formateo estructurado de salidas bajo los estándares de **Markdown (tablas estructuradas)** y **CSV (valores separados por comas)**.

Imaginemos que la dirección del **CEA Herman Ortiz Camargo** solicita un inventario urgente de las computadoras instaladas en el nuevo módulo educativo del **Barrio Saó**, detallando el estado operativo de los equipos donados para los laboratorios del tercer semestre.

Para automatizar este reporte sin que la IA agregue párrafos narrativos, se utiliza el siguiente diseño de prompt de formateo coercitivo:

Plaintext

```
[TAREA] Genera un inventario simulado de 3 servidores de red en el taller de  
computación del CEA Herman Ortiz Camargo, ubicado en el Barrio Saó.[FORMATO 1 -  
REQUERIDO] Devuelve la información organizada en una Tabla Markdown con  
alineación centrada.[FORMATO 2 - REQUERIDO] Inmediatamente abajo, proporciona  
exactamente los mismos datos estructurados en formato CSV nativo utilizando  
comas (,) como separadores de campos.[RESTRICCIÓN CRÍTICA] Está terminantemente
```



prohibido **escribir** palabras introductorias, saludos, consejos de configuración o texto explicativo. Solo imprime los bloques de formato solicitados.

El resultado de esta instrucción proporciona al Técnico la materia prima lista para ser copiada e importada directamente en sus respectivas herramientas de software:

Markdown

```
| Código del Servidor | Ubicación del Laboratorio | Memoria RAM  
Instalada | Estado de Funcionamiento || :  
---: / :  
---: / :  
---: / :  
---: || CEA-SRV-01 | Laboratorio Redes Saó | 32 GB DDR4 | Operativo al 100% ||  
CEA-SRV-02 | Taller Ensamblaje Central | 16 GB DDR4 | En Mantenimiento  
Preventivo || CEA-SRV-03 | Aula Virtual Sistemas | 64 GB DDR5 | Respaldo Fuera  
de Línea |
```

Fragmento de código

```
Codigo_Servidor,Ubicacion_Laboratorio,RAM_Instalada,Estado_FuncionamientoCEA-  
SRV-01,Laboratorio Redes Sao,32 GB DDR4,Operativo al 100%CEA-SRV-02,Taller  
Ensamblaje Central,16 GB DDR4,En Mantenimiento PreventivoCEA-SRV-03,Aula Virtual  
Sistemas,64 GB DDR5,Respaldo Fuera de Linea
```

2.6. INGENIERÍA DE PROMPTS BASADA EN ROLES ACADÉMICOS Y TÉCNICOS

A nivel avanzado, un prompt no es solo una instrucción, sino un mecanismo que altera la distribución de probabilidad de las respuestas del modelo de lenguaje. Cuando indicamos a la IA "Actúa como un compilador estricto del lenguaje PHP en su versión 8.2", el modelo de Inteligencia Artificial restringe su vocabulario latente, eliminando modismos, explicaciones conversacionales o sintaxis de otros ecosistemas informáticos.

En el contexto de los proyectos de emprendimiento que se gestan en los vecindarios de Pailón, definir roles hiper-específicos permite simular entornos de pruebas de software antes de gastar recursos en servidores reales. Si asignamos el rol de "Auditor de Seguridad de Base de Datos", la IA analizará nuestras consultas SQL buscando activamente vulnerabilidades contra ataques de inyección de código que pongan en riesgo la privacidad de las listas de clientes en los negocios de los barrios residenciales.



2.7. EL MÉTODO CHAIN-OF-THOUGHT (CADENA DE PENSAMIENTO) PARA DESCOMPOSICIÓN DE PROBLEMAS COMPLEJOS

La Cadena de Pensamiento (Chain-of-Thought) es una técnica avanzada de prompting que obliga a los modelos de Inteligencia Artificial a desglosar problemas lógicos complejos en pasos secuenciales intermedios antes de enunciar el veredicto o código final. Al incorporar la instrucción "*Piensa detalladamente paso a paso explicando tu razonamiento deductivo antes de programar*", la tasa de error algorítmico disminuye hasta en un 85% en problemas de alta complejidad matemática o logística.



Esta técnica es crucial cuando un Técnico se enfrenta a problemas de optimización logística de gran envergadura en Pailón, como coordinar las rutas de recolección de residuos o distribución de agua potable entre las familias de los barrios alejados como **San José, Ovidio Barberi y Jardines de Pailón**. Al obligar a la IA a razonar sobre distancias, estado de caminos y capacidades de carga de manera fragmentada, el algoritmo resultante es infinitamente más robusto y aplicable a la realidad social del municipio.



2.8. TÉCNICAS DE PROMPTING AUTÓNOMO Y META-COGNICIÓN PARA VALIDACIÓN DE CÓDIGO

El horizonte de la ingeniería de prompts se centra en los prompts de meta-cognición o "auto-corrección" (Self-Refine). Esta técnica consiste en diseñar un prompt compuesto donde la Inteligencia Artificial genera una primera versión del código, pero antes de entregar la respuesta, la procesa a través de un "filtro espejo" de evaluación crítica diseñado por el mismo usuario.

Un prompt estructurado de auto-corrección obliga a la IA a actuar como dos entidades separadas: el programador que crea la solución para una microempresa del **Barrio San Expedito**, y el tester de software implacable que busca activamente bucles infinitos, variables huérfanas o fugas de memoria. El resultado final entregado al Técnico pasa por un proceso de pre-depuración automatizado, garantizando un software estable, eficiente y adaptado para correr en las computadoras con recursos de hardware limitados que suelen tener los pequeños emprendimientos de nuestra región.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades en el Uso de la IA para Programación*

Mito Común sobre la Inteligencia Artificial	Realidad Técnica Demostrada en Computación
La Inteligencia Artificial es inteligente por sí misma y entiende los problemas del contexto de Pailón sin que se le den detalles geográficos o de negocio.	La IA es un modelo estadístico probabilístico que requiere contexto explícito, límites estrictos y ejemplos detallados para generar algoritmos que funcionen en la realidad local.
Para programar bien con Inteligencia Artificial, basta con escribir prompts largos con muchas palabras elegantes y técnicas del habla cotidiana.	La efectividad de un prompt radica en su estructura lógica, delimitación de restricciones operativas, asignación de roles definidos y control preciso de los formatos de salida.

❑ *Glosario de Términos Técnicos*

1. **Prompt Estructurado:** Instrucción técnica minuciosamente organizada por secciones (Rol, Contexto, Restricciones y Salida) que minimiza la ambigüedad en los modelos de Inteligencia Artificial.



2. **Few-Shot Prompting (Inyección I/O):** Técnica avanzada que consiste en alimentar al modelo con ejemplos reales de entradas y salidas esperadas para establecer un patrón lógico rígido de respuesta.
3. **Chain-of-Thought (Cadena de Pensamiento):** Estrategia de razonamiento dirigido que fuerza a la IA a resolver un problema algorítmico de manera secuencial y segmentada, evitando conclusiones erróneas automáticas.
4. **Pseudocódigo Strict (PSeInt):** Lenguaje de diseño de algoritmos con reglas semánticas y sintácticas rígidas que simula la estructura de la programación real, utilizado en entornos académicos de ingeniería de software.
5. **Output Coercitivo Formateado:** Técnica de restricción en el prompt que bloquea toda conversación o texto plano del modelo de lenguaje, forzándolo a responder únicamente en estructuras de datos procesables como tablas Markdown o arreglos CSV.

□ *Ponte a Prueba*

1. ¿Cuál es el beneficio técnico principal de aplicar la técnica de Few-Shot Prompting en el diseño de nuestros algoritmos?
 1. A) Que la IA responda mucho más rápido consumiendo menos internet en el módulo del CEA.
 2. B) Reducir drásticamente la alucinación del modelo al proveer patrones explícitos de datos de entrada y salida esperados.
 3. C) Permitir que el código fuente generado se conecte automáticamente a redes sociales sin programar.
4. **Respuesta Correcta: B**
2. Si deseamos importar un listado de clientes de un negocio en el Barrio Central Fátima directamente a Microsoft Excel sin errores de codificación, ¿qué formato de salida debemos exigir estrictamente en nuestro prompt estructurado?
 1. A) Texto conversacional amigable con viñetas decorativas y negritas.



2. B) Código ejecutable en formato C++ con librerías nativas de servidores.
3. C) Formato estructurado CSV utilizando comas como delimitadores de campos de datos.
4. **Respuesta Correcta: C**

3. ¿Qué sucede si dejamos un prompt abierto sin definir restricciones de programación negativas o de alcance tecnológico?

1. A) El modelo de IA asume que somos estudiantes avanzados y optimiza el uso de memoria RAM del equipo.
2. B) La IA introduce con total libertad creativa funciones obsoletas, librerías externas innecesarias o variables globales propensas a fallas lógicas.
3. C) La computadora del taller de computación del Barrio Saó se bloquea por un desbordamiento de datos de red.
4. **Respuesta Correcta: B**



VALORACIÓN



REFLEXIÓN TECNOLÓGICA, ÉTICA Y AMBIENTAL EN PAILÓN

El uso de la Inteligencia Artificial aplicada al desarrollo de sistemas informáticos no está exento de dilemas éticos y sociales que el Técnico debe ponderar críticamente. En primera instancia, la automatización del pensamiento lógico mediante la generación automatizada de prompts democratiza la creación de software para los micro-emprendimientos del **Barrio Central Fátima** o el **Barrio Saó**, permitiendo que negocios familiares accedan a herramientas tecnológicas avanzadas sin los costos prohibitivos del software privativo transnacional. Esto impulsa una justicia social económica local evidente.

No obstante, surge un severo riesgo asociado a la seguridad de los datos y la privacidad ciudadana. Al construir prompts lógicos en plataformas públicas de Inteligencia Artificial, muchos desarrolladores cometen la grave imprudencia de inyectar bases de datos reales con nombres, cédulas de identidad, deudas financieras u horarios de entrega de los ciudadanos de Pailón. Estos



datos son absorbidos por las corporaciones internacionales para entrenar futuros modelos, vulnerando las leyes de protección de privacidad y dejando expuesta la soberanía tecnológica de nuestras comunidades chiquitanas. Es un deber ético ineludible del Técnico limpiar y anonimizar absolutamente toda la información sensible antes de ingresarla a un prompt de IA.

Finalmente, debemos concientizar el impacto ecológico silencioso de esta tecnología. Cada consulta realizada a un gran modelo de lenguaje para corregir un algoritmo mal optimizado demanda un consumo energético colosal en los centros de datos globales, traducándose en huella de carbono y en un incremento alarmante del desecho electrónico (e-waste) a nivel mundial. Como profesionales formados en el **CEA Herman Ortiz Camargo**, la mayor contribución ecológica que podemos realizar es diseñar prompts estructurados altamente eficientes desde el primer intento. Al evitar la repetición infinita de consultas erróneas en el chat, optimizamos los recursos computacionales del planeta y extendemos el ciclo de vida útil de los equipos del taller informático, demostrando un compromiso real con el desarrollo tecnológico sostenible de Pailón.



PRODUCCIÓN



LABORATORIO GUIADO: SISTEMA DE FACTURACIÓN AUTOMATIZADO PARA FARMACIAS DE TURNO EN PAILÓN

Objetivo del Laboratorio: Diseñar y validar un prompt estructurado de nivel profesional que obligue a la IA a generar un algoritmo en pseudocódigo compatible con PSeInt para gestionar las ventas de emergencia en una farmacia ubicada en el **Barrio Central Fátima**, que realiza despachos nocturnos de medicamentos críticos hacia los pacientes vulnerables del **Barrio San Expedito**. El sistema debe procesar impuestos locales, evaluar restricciones de stock y exportar los resultados directamente a formato de datos CSV.

Instrucciones Paso a Paso en la Computadora:

Paso 1: Configuración del Entorno de Desarrollo



Encienda la computadora del taller de computación del módulo del CEA en el **Barrio Saó**. Abra su navegador web predilecto e ingrese a la interfaz de la Inteligencia Artificial generativa disponible para su clase. En otra ventana, ejecute el software educativo PSeInt en modo "Estricto" para estar listos para la validación del código.

Paso 2: Transcripción del Prompt de Ingeniería Estructurada

Copie íntegramente el siguiente prompt estructurado avanzado, asegurándose de no alterar ninguna de las restricciones lógicas planteadas, y péguelo en la consola de entrada del modelo de lenguaje:

Plaintext

```
Actúa como un Ingeniero de Software Senior y Compilador estricto de PSeInt.
Genera la solución algorítmica para el control de ventas nocturnas de una
farmacia en el Barrio Central Fátima de Pailón, que envía insumos al Barrio San
Expedito.REQUISITOS LÓGICOS DEL ALGORITMO:1. Declarar variables para:
nombre_medicamento (Cadena), precio_unitario (Real), cantidad_comprada (Entero),
es_afiliado_cea (Cadena: "
SI" o "
NO").2. Si es_afiliado_cea es igual a "
SI", aplicar un descuento fijo del 10% sobre el subtotal de la compra de
medicamentos por convenio social con el CEA Herman Ortiz Camargo.3. Si el
destino del despacho es el "
Barrio San Expedito" y la hora de la compra está en el rango nocturno (entre las
22:00 y las 06:00), sumar un recargo de 15 Bs. por concepto de logística de
transporte seguro en mototaxi.4. El algoritmo debe mostrar en pantalla el
Subtotal, el Descuento aplicado, el Recargo de transporte y el Total Final Neto
en Bolivianos (Bs.).RESTRICCIONES ERICTAS DE RESPUESTA:- PROHIBIDO agregar
textos introductorios, explicaciones conceptuales o comentarios de cortesía.- El
código en pseudocódigo debe tener comentarios detallados en cada una de sus
líneas utilizando doble barra (
//).- Al final de la respuesta, genera una sección llamada "[EXPORT_CSV]" que
contenga una simulación de 3 transacciones ejecutadas estructuradas
estrictamente en formato CSV con campos separados por comas.Procesa ahora bajo
estas reglas lógicas.
```

Paso 3: Análisis y Validación de la Respuesta de la IA

Ejecute la instrucción y observe cómo la Inteligencia Artificial procesa la orden limitándose a entregar las estructuras técnicas requeridas. El pseudocódigo entregado por el modelo debe lucir con el siguiente estándar de limpieza:



Delphi

```
// Algoritmo modular para la facturación de la farmacia del Barrio Central
FátimaAlgoritmo FacturacionFarmaciaPailon Definir nombreMedicamento,
esAfiliado Como Cadena;
// Declara las variables de texto para el insumo y la validación de afiliación
Definir precioUnitario, subtotal, descuento, recargo, totalFinal Como Real;
// Declara las variables numéricas de tipo real para el flujo financiero
Definir cantidadComprada, horaCompra Como Entero;
// Declara las variables de tipo entero para el volumen de compra y control de
tiempo Escribir "
Ingrese el nombre del medicamento requerido:";
// Solicita el nombre del insumo médico por pantalla Leer nombreMedicamento;
// Almacena el texto ingresado en la variable correspondiente Escribir "
Ingrese el precio unitario del medicamento (Bs.):";
// Solicita el costo financiero del producto Leer precioUnitario;
// Captura el valor decimal del precio Escribir "
Ingrese la cantidad de unidades que desea comprar:";
// Solicita el volumen de unidades de la transacción Leer cantidadComprada;
// Guarda el entero correspondiente a la cantidad Escribir "
El cliente es estudiante o afiliado al CEA? (SI/NO):";
// Consulta el convenio social de descuento Leer esAfiliado;
// Captura la respuesta de texto del operador Escribir "
Ingrese la hora de la transacción (formato 24 horas, solo entero 0 a 23):";
// Solicita el tiempo de control logístico Leer horaCompra;
// Almacena la hora de la transacción subtotal <- precioUnitario *
cantidadComprada;
// Ejecuta el cálculo aritmético inicial del subtotal bruto Si esAfiliado ==
"
SI" Entonces // Abre la condición lógica para evaluar el descuento por convenio
académico descuento <- subtotal * 0.10;
// Calcula el diez por ciento del descuento sobre el monto bruto anterior
Sino // Bloque alternativo si el cliente no pertenece a la institución educativa
descuento <- 0.0;
// Asigna un valor de cero Bolivianos al descuento de la factura FinSi //
Termina de manera formal la primera estructura de control condicional Si
horaCompra >= 22 O horaCompra <= 6 Entonces // Evalúa si la compra califica en
el rango de recargo de transporte nocturno recargo <- 15.0;
// Asigna el costo logístico de 15 Bs. por envío al Barrio San Expedito Sino
// Bloque si la transacción se realiza en horario diurno regular recargo
<- 0.0;
// Establece el costo de envío en cero Bolivianos FinSi // Cierra la
estructura condicional de tiempo de forma segura totalFinal <- (subtotal -
descuento) + recargo;
// Resuelve la ecuación matemática final para hallar el total neto Escribir "
--- FACTURA DIGITAL DE EMERGENCEIA PAILÓN -
--";
// Imprime el encabezado del reporte en pantalla Escribir "
```



```

Medicamento Despachado: ", nombreMedicamento;
// Muestra el nombre de la mercadería procesada    Escribir "
Subtotal Bruto: ", subtotal, " Bs.";
// Despliega el monto base de la transacción comercial    Escribir "
Descuento Aplicado CEA: ", descuento, " Bs.";
// Muestra el dinero ahorrado por el convenio del cliente    Escribir "
Recargo Envíos San Expedito: ", recargo, " Bs.";
// Informa el costo logístico nocturno del mototaxi    Escribir "
TOTAL NETO A PAGAR: ", totalFinal, " Bs.";
// Entrega el valor definitivo que debe abonar el usuarioFinAlgoritmo //
Finaliza por completo el algoritmo de facturación de emergencia

```

Paso 4: Copiado y Prueba en PSeInt

Copie todo el código fuente anterior generado por la IA en su pantalla. Péguelo dentro del editor PSeInt en su computadora. Haga clic en el botón verde "Ejecutar" para verificar que el flujo lógico funcione a la perfección. Realice simulaciones ingresando valores de compras nocturnas (ejemplo: hora 23) y verifique que el sistema sume correctamente los 15 Bs. de recargo logístico hacia el **Barrio San Expedito**.

Paso 5: Extracción de los Datos CSV de Producción

Verifique que la IA haya generado correctamente la sección coercitiva de datos simulados al final de su respuesta. El bloque debe verse exactamente así, listo para ser guardado en un bloc de notas con extensión .csv e importado en Excel:

Plaintext

```

[EXPORT_CSV]ID_Transaccion,Medicamento,Barrio_Destino,Hora,SubtotalBs,DescuentoBs,RecargoBs,TotalNetoBsTX-001,Amoxicilina,San
Expedito,23,80.00,8.00,15.00,87.00TX-002,Paracetamol,Central
Fatima,14,30.00,0.00,0.00,30.00TX-003,Ibuprofeno,San
Expedito,02,50.00,5.00,15.00,60.00

```

Al concluir este laboratorio, usted habrá desarrollado con éxito un flujo de trabajo profesional completo, gobernando el comportamiento lógico de la Inteligencia Artificial mediante ingeniería de instrucciones avanzada.



RECURSOS ADICIONALES

1. **Guía Oficial de Sintaxis de Markdown:** Documentación de referencia técnica para aprender a formatear texto científico, tablas centradas y bloques de código de software con realce de sintaxis de forma profesional.
2. **Repositorio Oficial y Manual de PSeInt:** Sitio web y guías oficiales de la herramienta de software educativo para descargar instaladores, validar la semántica estricta del pseudocódigo y estudiar la lógica algorítmica básica.
3. **Documentación de Git e Ingeniería de Prompts:** Manuales técnicos abiertos y repositorios comunitarios detallados para comprender cómo estructurar instrucciones de código sin alucinaciones, optimizando el consumo de recursos informáticos en modelos de lenguaje modernos.